

Efficient Nonlinear Model Predictive Path Integral Control for Stochastic Systems considering Input Constraints

Hannes Homburger¹, Stefan Wirtensohn¹ and Johannes Reuter¹

Abstract—This paper compares novel methods to efficiently include input constraints using the nonlinear Model Predictive Path Integral (MPPI) approach. The MPPI algorithm solves stochastic optimal control problems and is based on sampled trajectories. MPPI results from the physical path integral framework. Sample-based algorithms are characterized by the fact that they can be computed in parallel and offer the possibility to handle discontinuous dynamics and cost functions. However, using standard MPPI the input costs in the Lagrange term have to be chosen quadratic. This fact is unfavorable for various real applications. Further, in standard nonlinear model predictive control (NMPC) approaches hard box constraints on the control input trajectory can be treated directly. In this contribution, novel architectures based on integrator action are compared. The investigated input constraint MPPI controllers were tested on an autonomous self-balancing vehicle. Therefore both, simulation and real-world experiments are presented. This paper addresses the question of how the MPPI algorithm can be further developed to consider input box constraints. Videos of the self-balancing vehicle are available at: <https://tinyurl.com/mvn8j7vf>

I. INTRODUCTION

NMPC is an optimal feedback-control strategy based on repetitively online computing the optimal control input trajectory subject to the recent system state [1]. The first element of this optimal control sequence is applied to the system. Recently, a sample-based NMPC algorithm [2] was published, which is based on the physical framework of path integrals [3]. In this approach, the path integral over sampled trajectories is used to compute the optimal control input trajectory, providing an optimal solution concerning an input signal perturbed by additive white Gaussian noise (AWGN). Because in real controllers only a finite number of trajectories can be sampled, their information content is responsible for the optimizer's, and thus the controller's performance [4]. Recently, different methods, such as [5], [6], and [7] have been published to increase the information content of the drawn trajectories. However, using the standard MPPI algorithm, control input trajectory constraints are considered by either using an explicit quadratic cost [8] expression, implicitly by adding integrator action and a penalty term in the Lagrange cost function [9], or by clamping with Savitsky–Galoy filtering [2]. In this paper, novel methods are compared to consider input constraints using integrator action. In section II, the basics of the path integral control framework and the MPPI control algorithm are presented. In section III, the MPPI extensions to consider input constraints

efficiently are shown. To compare the algorithms, a control task is described in section IV. Section V presents the results of the MPPI control in both, simulation and full-scale experiment. Finally, section VI concludes the results of this paper and presents ideas for future work.

II. PATH INTEGRAL CONTROL

For a special class of stochastic optimal control problems (OCs), the corresponding nonlinear Hamilton-Jacobi-Bellman (HJB) equation can be transformed into a linear second-order partial differential equation [3]. This enables the application of the Feynman-Kac lemma [10] which states that the value function solving the HJB is equal to a special forward diffusion process, that can be simulated by drawing samples of stochastic trajectories. Beside policy iteration-based optimal control approaches [11], [12], and [13] in the field of reinforcement learning (RL), recently, the framework of path integrals has been used to solve stochastic OCs online in NMPC applications [2], [8], and [9]. In the following the basics of the MPPI algorithm according to [2] are described briefly.

A. Solution of Stochastic Optimal Control Problems based on Path Integrals

Considering the nonlinear discrete-time dynamics

$$\mathbf{x}_{k+1} = \mathbf{F}(\mathbf{x}_k, \mathbf{v}_k), \quad (1)$$

where the nonlinear transition function $\mathbf{F}$ maps the system state at time instance k denoted by $\mathbf{x}_k \in \mathbb{R}^{n_x}$ and the input to the system at time instance k denoted by $\mathbf{v}_k \in \mathbb{R}^{n_u}$ to the system state at the next time instance. The input to the system is given by

$$\mathbf{v}_k \sim \mathcal{N}(\mathbf{u}_k, \mathbf{\Sigma}),$$

where $\mathcal{N}(\mathbf{u}_k, \mathbf{\Sigma})$ denotes a Gaussian distribution with expectation value $\mathbf{u}_k = (u_1, u_2, \dots, u_{n_u})^\top \in \mathbb{R}^{n_u}$ and covariance matrix $\mathbf{\Sigma} \in \mathbb{R}^{n_u \times n_u}$. By defining $\mathbf{u}_k$ as the desired system input, we assume that the desired system input is disturbed with AWGN before it is applied to the dynamical system (1). The desired control input sequence over an $N \in \mathbb{N}$ step horizon is given by

$$\mathbf{U} = (\mathbf{u}_0, \mathbf{u}_1, \dots, \mathbf{u}_{N-1})$$

and a sequence of disturbed inputs is given by

$$\mathbf{V} = (\mathbf{v}_0, \mathbf{v}_1, \dots, \mathbf{v}_{N-1}),$$

¹ Institute of System Dynamics, HTWG Konstanz - University of Applied Sciences, 78462 Konstanz, Germany {hhomburger, stwitten, jreuter} @htwg-konstanz.de

that is a random sequence with the corresponding probability density function

$$q(\mathbf{V}|\mathbf{U}, \mathbf{\Sigma}) = \prod_{k=0}^{N-1} [(2\pi)^{n_u} |\mathbf{\Sigma}|]^{-\frac{1}{2}} \exp\left(-\frac{1}{2} \mathbf{\epsilon}_k^\top \mathbf{\Sigma}^{-1} \mathbf{\epsilon}_k\right),$$

where $\mathbf{\epsilon}_k = \mathbf{v}_k - \mathbf{u}_k$ denotes the sampled input disturbance at time instance k . Analogous to the approach of direct single shooting according to [1], an initial state $\mathbf{x}_0$ and a sequence of disturbed inputs $\mathbf{V}$ are deterministically mapped to a state trajectory by applying (1) recursively

$$\begin{pmatrix} \tilde{\mathbf{x}}_0(\mathbf{x}_0, \mathbf{V}) \\ \tilde{\mathbf{x}}_1(\mathbf{x}_0, \mathbf{V}) \\ \vdots \\ \tilde{\mathbf{x}}_N(\mathbf{x}_0, \mathbf{V}) \end{pmatrix} = \mathbf{X}(\mathbf{x}_0, \mathbf{V}) = \begin{pmatrix} \mathbf{x}_0 \\ \mathbf{F}(\tilde{\mathbf{x}}_0, \mathbf{v}_0) \\ \vdots \\ \mathbf{F}(\tilde{\mathbf{x}}_{N-1}, \mathbf{v}_{N-1}) \end{pmatrix},$$

where $\mathbf{X} : \mathbb{R}^{n_x} \times \mathbb{R}^{n_u \times N} \rightarrow \mathbb{R}^{n_x \times (N+1)}$ denotes the operator describing the recursive simulation. Note that each element $\tilde{\mathbf{x}}_k$ is only dependent on the first k elements in $\mathbf{V}$. The discrete time stochastic OCP is defined by

$$\min_{\mathbf{U}} \mathbb{E}_{\mathbf{V} \sim \mathcal{Q}_{\mathbf{U}, \mathbf{\Sigma}}} \left[\sum_{k=0}^{N-1} L(\tilde{\mathbf{x}}_k(\mathbf{x}_0, \mathbf{V}), \mathbf{u}_k) + E(\tilde{\mathbf{x}}_N(\mathbf{x}_0, \mathbf{V})) \right], \quad (2)$$

where $L : \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \rightarrow \mathbb{R}^+$ denotes the Lagrange term, $E : \mathbb{R}^{n_x} \rightarrow \mathbb{R}^+$ denotes the Mayer term, and $\mathbb{E}_{\mathcal{Q}}$ denotes the expectation operator with

$$\mathbb{E}_{\mathbf{V} \sim \mathcal{Q}_{\mathbf{U}, \mathbf{\Sigma}}} [\phi(\mathbf{V}, \mathbf{U}, \mathbf{\Sigma})] = \int \phi(\mathbf{V}, \mathbf{U}, \mathbf{\Sigma}) q(\mathbf{V}|\mathbf{U}, \mathbf{\Sigma}) d\mathbf{V}, \quad (3)$$

where ϕ denotes a scalar function. Using the path integral framework, the Lagrange cost term

$$L(\tilde{\mathbf{x}}_k(\mathbf{x}_0, \mathbf{V}), \mathbf{u}_k) = l(\tilde{\mathbf{x}}_k(\mathbf{x}_0, \mathbf{V})) + \frac{1}{2} \mathbf{u}_k^\top \mathbf{R} \mathbf{u}_k \quad (4)$$

consists of two terms, $l : \mathbb{R}^{n_x} \rightarrow \mathbb{R}^+$ denotes state costs which are only dependent on the system state, and the other term is a quadratic function of the control inputs, penalized with $\mathbf{R} := \lambda \mathbf{\Sigma}^{-1}$, where $\lambda \in \mathbb{R}^+$ is called the *inverse temperature*. This restriction is necessary to apply the path integral framework. For further information, the reader is referred to [2] and [14]. Summing up all state-dependent costs of a trajectory yields the composition

$$S(\mathbf{x}_0, \mathbf{V}) = \sum_{k=0}^{N-1} l(\tilde{\mathbf{x}}_k(\mathbf{x}_0, \mathbf{V})) + E(\tilde{\mathbf{x}}_N(\mathbf{x}_0, \mathbf{V}))$$

called the *path costs*. Using these expressions [2] shows that each element of the optimal input control sequence (2) is given by

$$\mathbf{u}_k^* = \mathbb{E}_{\mathbf{V} \sim \mathcal{Q}_{\hat{\mathbf{U}}, \mathbf{\Sigma}}} [\omega(\mathbf{V}) \mathbf{v}_k], \quad (5)$$

where $\hat{\mathbf{U}} = (\hat{\mathbf{u}}_0, \hat{\mathbf{u}}_1, \dots, \hat{\mathbf{u}}_{N-1})$ denotes a guess of the minimizer of (2) and the *importance weighting* is given by

$$\omega(\mathbf{V}) = \frac{1}{\eta} \exp\left(-\frac{1}{\lambda} \left(S(\mathbf{x}_0, \mathbf{V}) + \frac{1}{2} \sum_{k=0}^{N-1} (2\mathbf{v}_k - \hat{\mathbf{u}}_k)^\top \mathbf{R} \hat{\mathbf{u}}_k \right)\right)$$

with the normalization constant

$$\eta = \int \exp\left(-\frac{1}{\lambda} \left(S(\mathbf{x}_0, \mathbf{V}) + \frac{1}{2} \sum_{k=0}^{N-1} (2\mathbf{v}_k - \hat{\mathbf{u}}_k)^\top \mathbf{R} \hat{\mathbf{u}}_k \right)\right) d\mathbf{V}$$

used to ensure $\int \omega(\mathbf{V}) d\mathbf{V} = 1$. To compute the optimal control, (5) is approximated using the Monte-Carlo estimation

$$\hat{\mathbf{u}}_k^* = \sum_{m=1}^M \omega(\mathbf{V}^m) \mathbf{v}_k^m \quad (6)$$

of the sampled disturbed input sequences

$$\mathbf{V}^m = (\mathbf{v}_0^m, \mathbf{v}_1^m, \dots, \mathbf{v}_{N-1}^m), \quad m = 1, 2, \dots, M.$$

Note, for $M \rightarrow \infty$ (6) converges to (5). Further, the definition $\mathbf{v}_k^m := \hat{\mathbf{u}}_k + \mathbf{\epsilon}_k^m$ is given for $k = 0, \dots, N-1$. In algorithm 1 the resulting path integral control optimizer is presented.

Algorithm 1 Path Integral Optimizer (PIO)

Input: $\mathbf{F}$: System dynamics;

M : Number of sampled trajectories;

N : Length of prediction horizon;

$\mathbf{U}^I$: Initial control sequence;

$\mathbf{x}_0$: Recent system state;

l, E : State costs and Mayer term;

$\mathbf{\Sigma}, \lambda$: Covariance and inverse temperature;

Output: $\mathbf{U}^O$: Optimized control sequence

$\bar{S}$: Average costs;

```

1: for  $m \in \{1, 2, \dots, M\}$  do
2:    $\tilde{\mathbf{x}}_0^m \leftarrow \mathbf{x}_0$ ;
3:   Sample  $\{\mathbf{\epsilon}_0^m, \mathbf{\epsilon}_1^m, \dots, \mathbf{\epsilon}_{N-1}^m\}$ ;
4:    $S_m \leftarrow 0$ ;
5:   for  $k \in 0, 1, \dots, N-1$  do
6:      $\tilde{\mathbf{x}}_{k+1}^m \leftarrow \mathbf{F}(\tilde{\mathbf{x}}_k^m, \mathbf{u}_k^I + \mathbf{\epsilon}_k^m)$ ;
7:      $S_m \leftarrow S_m + l(\tilde{\mathbf{x}}_k^m) + \frac{\lambda}{2} (\mathbf{u}_k^I + 2\mathbf{\epsilon}_k^m)^\top \mathbf{\Sigma}^{-1} \mathbf{u}_k^I$ ;
8:   end for
9:    $S_m \leftarrow S_m + E(\tilde{\mathbf{x}}_N^m)$ ;
10: end for
11:  $\eta \leftarrow \sum_{m=1}^M \exp(-\frac{1}{\lambda} S_m)$ ;
12: for  $m \in \{1, 2, \dots, M\}$  do
13:    $\omega_m \leftarrow \frac{1}{\eta} \exp(-\frac{1}{\lambda} S_m)$ ;
14: end for
15: for  $k \in \{0, 1, \dots, N-1\}$  do
16:    $\mathbf{u}_k^O \leftarrow \sum_{m=1}^M \omega_m \mathbf{v}_k^m$ ;
17: end for
18: return  $\mathbf{U}^O = \{\mathbf{u}_0^O, \mathbf{u}_1^O, \dots, \mathbf{u}_{N-1}^O\}$ 

```

B. Information-Theoretic Model Predictive Control

In the following the MPPI algorithm is described which is a stochastic NMPC algorithm based on the introduced path integral optimizer. Due to Bellman's curse of dimensions and the representation problem, only in special cases, it is possible to represent and compute the optimal control law $\mathbf{u}_k^*(\mathbf{x}_k)$ offline. However, due to the independence of the for-loop iterations in algorithm 1 lines 1-10, it can efficiently be computed in parallel tasks. Due to model-plant mismatch and unexpected disturbances the optimal control

input sequence is calculated repetitively in the NMPC framework. The only difference between the stochastic OCPs to be solved repetitively is the recent system state $\mathbf{x}_0$. Therefore, the controller's performance can be significantly increased by shifting the control sequence and warm-starting. Those are well known methods to get a good proposal density $q(\mathbf{V}|\hat{\mathbf{U}}, \Sigma)$ [4]. Thus, the MPPI algorithm can be described as the following four steps, which are repeated cyclically:

- 1) Get recent system state $\mathbf{x}_0$
- 2) Optimize $\mathbf{U} \leftarrow \text{PIO}(\mathbf{U})$ using algorithm 1
- 3) Apply the first element $\mathbf{u}_0$
- 4) Shift $\mathbf{U}$ by $\mathbf{u}_k \leftarrow \mathbf{u}_{k+1}$, $k = 0, 1, \dots, N-2$

Recently, different full-scale applications have shown that this control approach is real-time capable and can handle nonlinear stochastic systems and discontinuous cost functions e.g. to drive aggressively with an autonomous model car [2] or to autdock a vessel [9]. In most standard NMPC approaches, input constraints can be considered directly within the transcription and added to the resulting nonlinear problem as inequality constraints on the decision variables. However, using the standard MPPI algorithm according to [2], it is only permitted to penalize high control input values with quadratic input costs used in (4). In the next section, we present and compare different approaches to consider input constraints in the MPPI scheme.

III. INPUT CONSTRAINED MPPI CONTROL

The stochastic OCP we want to solve using input constrained MPPI control is defined as

$$\bar{\mathbf{U}}^* = \underset{\bar{\mathbf{U}} \in \mathcal{U}}{\operatorname{argmin}} \mathbb{E}_{\mathbf{V} \sim \mathbb{Q}_{\bar{\mathbf{U}}, \Sigma}} \left[\sum_{k=0}^{N-1} L(\bar{\mathbf{x}}_k(\mathbf{V}), \bar{\mathbf{u}}_k) + E(\bar{\mathbf{x}}_N) \right], \quad (7)$$

where $\mathcal{U}$ denotes the feasible set of the control input sequences. The definition

$$\mathcal{U} = \{\mathbf{U} | \mathbf{u}^- \leq \mathbf{u}_k \leq \mathbf{u}^+, \forall k \in \{1, \dots, N\}\},$$

where the inequalities are evaluated element-wise, the input control vector's lower bound is denoted by $\mathbf{u}^- \in \mathbb{R}^{n_u}$, and the control input vector's upper bound is denoted by $\mathbf{u}^+ \in \mathbb{R}^{n_u}$ with element-wise $\mathbf{u}^- \leq \mathbf{u}^+$, enables to treat box constraints on the control input sequence. The validity of the path integral framework is based on quadratic input costs (4) and no restrictions on the feasible control set. Thus, we can use the idea of applying integrator action with the extension of the state space

$$\bar{\mathbf{x}}_k = \begin{bmatrix} \mathbf{x}_k \\ \mathbf{w}_k \end{bmatrix},$$

where $\mathbf{w}_k \in \mathbb{R}^{n_u}$ are the states of the integrator at time instance k , thus the extended dynamics is given by

$$\bar{\mathbf{x}}_{k+1} = \bar{\mathbf{F}}(\bar{\mathbf{x}}_k, \mathbf{v}_k) = \begin{bmatrix} \mathbf{F}(\mathbf{x}_k, \mathbf{w}_k) \\ \mathbf{I}(\mathbf{w}_k, \mathbf{v}_k) \end{bmatrix},$$

where $\mathbf{I}$ denotes a discrete time integrator of the dimension n_u . Thus, the control input vector of the real system has become part of the state vector of an expanded system. Regarding the expanded system dynamics, the following two

properties of the path integral framework described in [2] can be exploited to solve (7):

- 1) The state costs l in (4) can be chosen arbitrarily, thus the input box constraints can be considered by adding a nonlinear penalty term.
- 2) The dynamics $\mathbf{F}$ defined in (1) can be discontinuous, thus we can saturate the integrator state $\mathbf{w}_k$ due to its artificial nature.

These two methods will be explained in the following.

A. Input Constrained MPPI Control using Penalty Terms

Similar to merit functions, known for globalization in standard NMPC approaches we can define the state-dependent costs as

$$l(\bar{\mathbf{x}}_k) = \bar{l}(\mathbf{x}_k) + \sigma \left(\min_{\mathbf{u}^- \leq \hat{\mathbf{u}} \leq \mathbf{u}^+} \|\mathbf{w}_k - \hat{\mathbf{u}}\|_1 \right), \quad (8)$$

where $\sigma \in \mathbb{R}^+$ denotes the penalty coefficient. Note, if and only if $\mathbf{u}^- \leq \mathbf{w}_k \leq \mathbf{u}^+$ holds, the box constraints are satisfied and the penalty term vanishes. For implementation due to the convexity of the minimization, an element-wise comparison causes only low computational effort. This approach is called penalty term (PT) in the rest of the paper.

B. Input Constrained MPPI Control using Saturation

According to [15], another method to consider input inequality constraints using MPPI control is to use the saturated integrator structure substituting $\mathbf{I}(\mathbf{w}_k, \mathbf{v}_k)$ by

$$\mathbf{w}_{k+1} = \bar{\mathbf{I}}(\mathbf{w}_k, \mathbf{v}_k) = \begin{bmatrix} \bar{I}_1(w_{1,k}, v_{1,k}) \\ \bar{I}_2(w_{2,k}, v_{2,k}) \\ \vdots \\ \bar{I}_{n_u}(w_{n_u,k}, v_{n_u,k}) \end{bmatrix}, \quad (9)$$

with the $n \in \{1, 2, \dots, n_u\}$ saturated scalar integrators

$$\bar{I}_n(w_n, v_n) = \begin{cases} u_n^+ & \text{if } w_n + hv_n > u_n^+ \\ u_n^- & \text{if } w_n + hv_n < u_n^- \\ w_n + hv_n & \text{else} \end{cases},$$

where w_n , v_n , u_n^+ , and u_n^- denote the n -th element of the corresponding vectors, and $h > 0$ denotes the sample time of the system. This approach is called saturated integrator (SI) in the rest of the paper. Note that this integrator is the saturated version of the integrator with elements $I_n(w_n, v_n) = w_n + hv_n$ used in the PT approach. While using the PT approach, the input constraints can be violated. Using the SI approach, the input inequality constraints are satisfied securely. Since the computational cost of solving the minimization problem in (8) and considering the saturation in the integrator (9) require negligible computational effort, the two approaches do not differ in computational complexity.

IV. APPLICATION EXAMPLE

The advantages and disadvantages of the presented methods are investigated using experiments in an application scenario. Specifically, positioning control of a self-balancing vehicle is considered. In this application, due to safety reasons the desired velocity has to be limited strictly thus



Fig. 1. Photography of the self-balancing test vehicle *MonoChair2* in full-scale at the forum of the HTWG Konstanz - University of Applied Sciences.

standard quadratic penalty is not possible. The self-balancing vehicle is shown in Fig. 1. In this section, the dynamics, the inequality input constraints, the cost functions, and the information-theoretic parameters are presented which are combined to formulate an input constraint stochastic OCP in the form of (7). It should be noted that linear controllers have already been implemented for the balancing task, the velocity control, and the yaw control. These simplify the model and reduce the influence of unpredictable disturbances.

A. Vehicle Dynamics

The state of the controlled self-balancing vehicle is given by

$$\mathbf{x} = (x, y, \gamma, c, r)^\top,$$

where x , y , and γ denote the generalized local east-north-up pose, and the body-fixed surge velocity is given by $v = ar + c$. Note that r and c represent the internal states of the closed velocity control loop (10) given by

$$G_v(s) = \frac{as + 1}{\frac{1}{\Omega^2}s^2 + \frac{2\zeta}{\Omega}s + 1}, \quad (10)$$

where s denotes the complex frequency variable, $a < 0$ denotes a constant, that is related to the non-minimum phase zero of the system, $\Omega > 0$ denotes the radial frequency, and the damping of the system is denoted by $\zeta > 0$. The transfer model of the closed yaw control loop is modeled using the first-order transfer function

$$G_\gamma(s) = \frac{1}{T_e s + 1},$$

where $T_e > 0$ denotes the equivalent time constant. Defining the reference values of the subordinated control loops yields

TABLE I
MODEL PARAMETERS OF SELF-BALANCING VEHICLE

P	Value	P	Value	P	Value
a	$-0.32 \text{ (rad/s)}^{-1}$	ζ	0.92	v_{des}^+	0.5 m/s
Ω	0.89 rad/s	T_e	2 s	v_{des}^-	0 m/s

the system input

$$\mathbf{u} = \begin{pmatrix} v_{\text{des}} \\ \gamma_{\text{des}} \end{pmatrix}$$

and the nonlinear continuous time system dynamics

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}) = \begin{pmatrix} c \cos \gamma + ar \cos \gamma \\ c \sin \gamma + ar \sin \gamma \\ -\frac{1}{T_e} \gamma + \frac{1}{T_e} \gamma_{\text{des}} \\ r \\ -\Omega^2 c - 2\zeta \Omega r + \Omega^2 v_{\text{des}} \end{pmatrix} \quad (11)$$

in the state space representation. The nonlinear time continuous system dynamics can be transformed into the nonlinear discrete time solution map given in (1). Therefore, the Runge-Kutta 4th order method (RK4) with step size $h_{\text{RK}} > 0$ is applied. The parameters of the used system model are shown in Table I. Because the input variables enter linearly the dynamics of the velocity and the yaw angle, stochastic uncertainties corresponding to these states due to unmodeled effects (e.g. the cobblestones shown in Figure 1) can be modeled by the uncertainty of the input variables. As an assumption, these effects are modeled with Gaussian noise.

B. Inequality Input Constraints

As already mentioned, for safety reasons the desired velocity has to be limited to the range

$$v_{\text{des}}^- \leq v_{\text{des}}(t) \leq v_{\text{des}}^+, \quad t \in [0, T],$$

where $T > 0$ denotes the prediction horizon. Due to the zero-order hold property, the defined input constraints can be checked by checking all elements of the control input sequence for compliance with

$$v_{\text{des}}^- \leq v_k^{\text{des}} \leq v_{\text{des}}^+, \quad k \in \{0, 1, \dots, N-1\},$$

where $v_k^{\text{des}} = v_{\text{des}}(kh)$, $N = \lfloor T/h \rfloor$, and $k = 0$ denote the initialization of the MPC iteration. Note, this limitation defines the feasible set $\mathcal{U}$ used in (7). It is not necessary to limit the desired yaw orientation, because the modulo property $\gamma_{\text{des}} = \gamma_{\text{des}} + 2n\pi$, $n \in \mathbb{Z}$ holds for this state. In the next part, the definition of the cost functions for the positioning control task is given.

C. Cost Functions and Information-Theoretic Parameters

Using MPPI control only the state-dependent term l of the Lagrange costs (4) can be defined to achieve the desired behavior. Acc. to [9] first linguistic quality criteria are defined, then translated into math terms, which are summed up to get the state-dependent cost function. In our task we only need to define one linguistic quality criterion: *It is*

TABLE II
MPPI CONTROL AND COST PARAMETERS

P	Value	P	Value	P	Value	P	Value
h	0.5 s	x_{des}	10 m	a_l	1000 m ⁻¹	σ_v	0.005
h_{RK}	0.1 s	y_{des}	5 m	a_E	1000 m ⁻¹	σ_γ	0.0035
T	10 s	λ	1	Σ	diag($\sigma_v^2, \sigma_\gamma^2$)	σ	50000

considered good to be next to the 2d position $(x_{\text{des}}, y_{\text{des}})$. While there are a lot of different methods to penalize the residual to this desired pose, the easiest approach is to use the L1-norm

$$l(\mathbf{x}) = a_l \| (x_{\text{des}}, y_{\text{des}})^\top - (x, y)^\top \|_1, \quad (12)$$

where $a_l > 0$ denotes a weighting coefficient. Because no further criteria were defined, the Mayer term with $E(\mathbf{x}_N) = a_E \| (x_{\text{des}}, y_{\text{des}})^\top - (x_N, y_N)^\top \|_1$, where $a_E > 0$ is chosen similar to the state-dependent cost function of the Lagrange term. After defining the system dynamics, the input constraints, and the cost functions, only the definition of the information-theoretic parameters given by the covariance matrix Σ and the *inverse temperature* λ must be selected concerning the experiments. Remember, the product $\lambda \Sigma^{-1}$ provides the weighting of the quadratic control costs denoted by $\mathbf{R}$ in the path integral control framework [3].

V. RESULTS

In this section, the results of the experiments are shown that have been performed in simulation and full-scale to compare the performances of the different input constrained MPPI approaches presented in Section 3.

A. Simulation Results

To test the performances in simulation, the controllers and the dynamic model (11) are embedded in a simulation environment. The used values of the controller parameters (P) are given in Table II for reproducibility. To simulate the

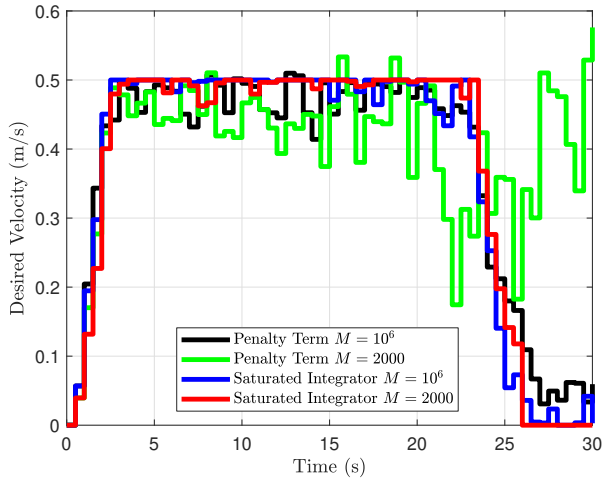


Fig. 2. Sequences of the desired velocity in simulation experiments with different setups.

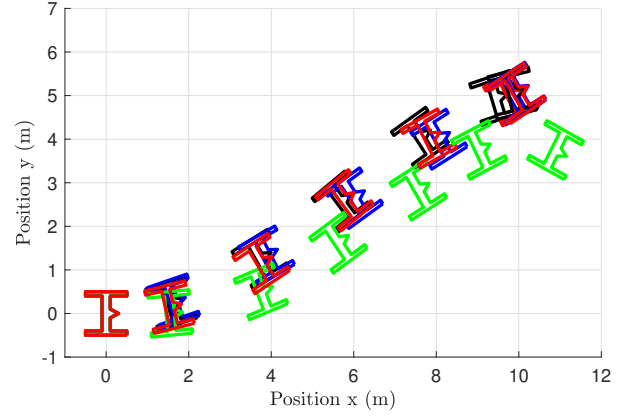


Fig. 3. Generalized pose at the time instances $k\delta T$ with $k \in \{0, 1, \dots, 6\}$ and $\delta T = 5$ s of the PT approach with $M = 10^6$ (black), $M = 2000$ (green) and the SI approaches with $M = 10^6$ (blue), $M = 2000$ (red) in simulation.

time continuous system an RK4 with a step size of 5 ms is used. The initial state is given by $\bar{\mathbf{x}}_0 = \mathbf{0}$. The simulation time is set to 30 s, which is appropriate for the given task. To obtain a benchmark, both methods are applied with $M = 10^6$ trajectories. In addition, $M = 2000$ trajectories are applied, which is a real-time capable setting in full-scale. Due to the high computational effort, this setting is only applicable in simulation. The resulting generalized positions are shown in Fig. 3. Note, the orientation is important because it affects the dynamics (11). Using the SI setup for both tests independent of the number of drawn trajectories the autonomous vehicle reaches the desired position and brakes as required. Using the PT setup only in the experiment with the high number of drawn trajectories the aim position is reached. In the progress of the simulation, the PT setup with $M = 2000$ does not steer enough and is slower than the other setups. The steering behavior is an interesting effect because the different setups do not change the yaw control loop directly. However, the exploration of the state space by sampling trajectories is done

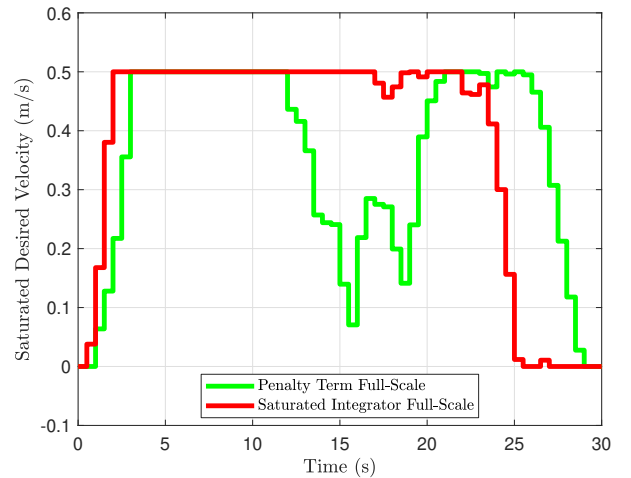


Fig. 4. Sequences of the limited desired velocity in full-scale experiments using the different methods.

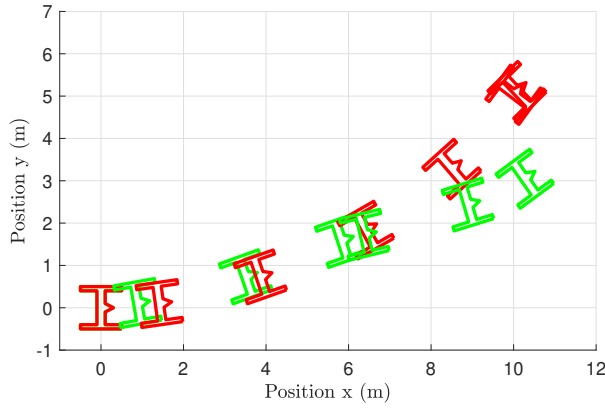


Fig. 5. Generalized pose at the time instances $k\delta T$ with $k \in \{0, 1, \dots, 6\}$ and $\delta T = 5$ s of the PT approach with $M = 2000$ (green) and the SI approach with $M = 2000$ (red) in full-scale experiments.

simultaneously for both system inputs. Thus, trajectories having good steering behavior but violating the velocity constraints are penalized and have only a minimal effect on the path integral solution (6). Figure 2 shows the sequence of applied desired velocities. While the SI methods consider the inequality constraints excellently at all time instances, the PT methods overshoot at some time instances. Further, the PT methods do not use the whole feasible set and are oscillating around some value next to v_{des}^+ for $2.5 \text{ s} < t < 24 \text{ s}$. This is a direct result of the PT method's design to avoid high cost states.

B. Full-Scale Results

To test the performance of the MPPI controller in the full-scale experiment, the MPPI controller is applied on the self-balancing test vehicle *MonoChair2* shown in Fig. 1. For this purpose, the MPPI controller is implemented on the NVIDIA® Jetson Nano™ Developer Kit and connected to the system with Ethernet. Then, the same experiments as in the previous section are conducted, although only $M = 2000$ can be drawn due to the computational effort. While Fig. 5 shows the generalized pose during the experiments, the corresponding desired velocity sequences are shown in Fig. 4. The SI approach in full-scale shows similar behavior as in simulation in both, the evolution of the generalized pose and the sequence of desired velocities. Due to the plant-model mismatch, the PT approach brakes at $t = 11.5 \text{ s}$. This further deteriorates the behavior. An appropriate criterion to compare the performances quantitatively is the accumulated Lagrange cost (12) over 30 s. This yields $J_{\text{PT},\text{sim}} = 478$, $J_{\text{SI},\text{sim}} = 418.7$, $J_{\text{PT},\text{full}} = 522.8$ and $J_{\text{SI},\text{full}} = 425.3$, showing that SI is the recommended approach because it performs better in both scenarios.

VI. CONCLUSION AND FUTURE WORK

This paper presents and compares two novel methods considering input constraints using MPPI control. In both methods, the control sequence is low-pass filtered by the expanded integrators, leading to a comfortable behavior. In

the simulation and in the full-scale experiments, the controlled system shows excellent performance even under the influence of model-plant-mismatch and disturbances. While both methods can consider the input inequality constraints by drawing $M = 10^6$ trajectories in the benchmark, in applications the SI method is recommended. Concerning other NMPC approaches, the MPPI control algorithm is characterized by very good parallelizability and the possibility to handle non-differentiable dynamics and cost functions. In future work, it would be interesting to proof the stability of systems using the SI MPPI approach and apply it to solve stochastic OCPs representing more complex tasks.

ACKNOWLEDGMENT

Special thanks to Matthias Albrecht for supporting the full-scale experiments.

REFERENCES

- [1] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed. Santa Barbara, CA, USA: Nob Hill, 2017.
- [2] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, *Information-Theoretic Model Predictive Control: Theory and Applications to Autonomous Driving*. *IEEE Transactions on Robotics*, vol. 34, no. 6, 2018, pp. 1603–1622.
- [3] H. J. Kappen, Path integrals and symmetry breaking for optimal control theory, In *J. Statist. Mech., Theory Exp.*, vol. 2005, no. 11, 2005, Art. no. P11011.
- [4] S. Thijssen and H. J. Kappen, Path Integral Control and State Dependent Feedback. In *Phys. Rev. E* 91, 032104, 2015.
- [5] I. M. Balci, E. Bakolas, B. Vlahov and E. A. Theodorou, Constrained Covariance Steering Based Tube-MPPI. In *Proc. of the ACC 2022 American Control Conference (ACC)*, Atlanta, USA.
- [6] H. Homburger, S. Wirtensohn, M. Diehl and J. Reuter, Feature-Based Proposal Density Optimization for Nonlinear Model Predictive Path Integral Control. In *Proc. of the 6th IEEE Conference on Control Technology and Application (CCTA)*, Trieste, Italy, 2022.
- [7] R. Kusumoto, L. Palmieri, M. Spies, A. Csiszar and K. O. Arras, Informed Information Theoretic Model Predictive Control. In *Proceedings of IEEE 2019 International Conference on Robotics and Automation (ICRA)*, Montreal, Canada, 2019, pp. 2047–2053.
- [8] V. Gomez, S. Thijssen, A. Symington, S. Hailes and H. J. Kappen, Real-Time Stochastic Optimal Control for Multi-agent Quadrotor Systems. In *26th International Conference on Automated Planning and Scheduling (ICAPS 16)*, London, Great Britain, 2015.
- [9] H. Homburger, S. Wirtensohn and J. Reuter, Docking Control of a Fully-Actuated Autonomous Vessel using Model Predictive Path Integral Control. In *Proc. of the 20th European Control Conference (ECC)*, London, Great Britain, 2022, pp. 755–760.
- [10] M. Kac, On distributions of certain Wiener functionals, *Transactions of the American Mathematical Society*, vol. 65, 1949, no. 1, pp. 1–13.
- [11] E. A. Theodorou, J. Buchli und S. Schaal, A Generalized Path Integral Control Approach to Reinforcement Learning. In *Journal of Machine Learning Research*, Vol. 11, 2010, pp. 3137–3181.
- [12] H. J. Kappen and H. C. Ruiz, Adaptive Importance Sampling for Control and Inference. In *Journal of Statistical Physics*, vol. 162, 2016, pp. 1244–1266.
- [13] T. Lefebvre and G. Crevecoeur, Path Integral Policy Improvement with Differential Dynamic Programming. In *Proc. of the 2019 IEEE/ASME International Conference on Advanced Intelligent Mechatronics Hong Kong, China, July 8-12, 2019*, pp. 739–745.
- [14] E. A. Theodorou, Nonlinear stochastic control and information theoretic dualities: Connections, interdependencies and thermodynamic interpretations, *Entropy*, vol. 17, no. 5, 2015, pp. 3352–3375.
- [15] T. Kim, G. Park, K. Kwak, J. Bae, and W. Lee, Smooth Model Predictive Path Integral Control Without Smoothing, *IEEE Robotics and Automation Letters*, vol. 7, no. 4, 2022, pp. 10406–10413.